

Message Intelligence System

Spam Detection using Machine Learning

KNN · SVM · Naive Bayes

Project Report | June 23, 2026

Dataset	Message Intelligence Dataset
Total Records	5,200 messages
Target Variable	spam_label (0 = Legitimate, 1 = Spam)
Models	K-Nearest Neighbors, SVM, Naive Bayes
Tools	Python, scikit-learn, pandas, matplotlib

1. Executive Summary

This report presents a comprehensive machine learning project to classify messages as **Spam** or **Legitimate** using the Message Intelligence Dataset (5,200 records). Three classifiers were implemented and evaluated: KNN, SVM, and Naive Bayes.

5,200	81.3%	18.7%	3	100%
Total Messages	Legitimate	Spam	Models Trained	Best Accuracy

Class Distribution (5,200 Messages)

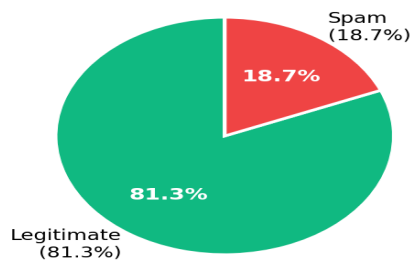


Fig 1. Class Distribution of Dataset

Train / Test Split (80% / 20%)

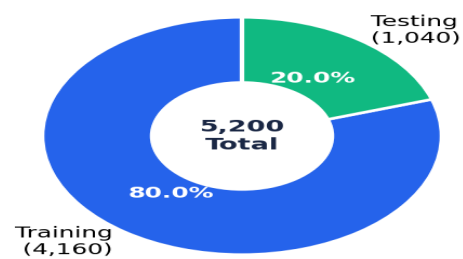


Fig 2. Train / Test Split (80% / 20%)

2. Theoretical Foundation (Part A)

2.1 Conditional Probability & Bayes' Theorem

Conditional Probability $P(A|B) = P(A \cap B) / P(B)$ is the foundation for Bayes' Theorem: $P(A|B) = P(B|A) \times P(A) / P(B)$. This underpins the Naive Bayes classifier used in spam detection.

2.2 Classifier Comparison

Feature	KNN	Naive Bayes	SVM
Type	Distance-Based	Probabilistic	Margin-Based
Training Speed	Very Fast	Very Fast	Moderate
Pred. Speed	Slow	Fast	Fast
Scaling Needed	Yes	No	Yes
Best Use Case	Small datasets	Text / Spam	Complex problems

3. Dataset Overview & Preparation (Part B)

3.1 Dataset Structure — 5,200 rows × 16 columns

Column	Type	Description
message_length	int64	Total character count
word_count	int64	Number of words
num_urls	int64	Count of URLs in message
num_digits	int64	Count of digit characters
num_special_chars	int64	Count of special characters
spam_keyword_score	int64	Score from spam-related keywords
legit_keyword_score	int64	Score from legitimate keywords
sender_activity_score	float64	Sender activity (106 NaN values)
sender_account_age_days	float64	Account age in days (113 NaN)
messages_sent_last_24h	float64	Messages sent last 24h (162 NaN)
hour_of_day	int64	Hour message was sent (0–23)
day_of_week	int64	Day of week (0=Monday)
spam_label	int64	Target: 0=Legitimate, 1=Spam

3.2 Missing Values

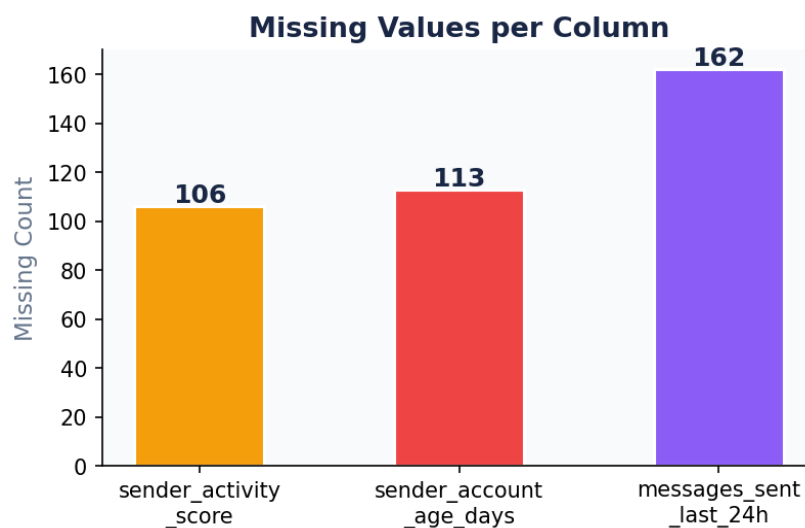


Fig 3. Missing Values per Column (filled with median)

3.3 Feature Importance

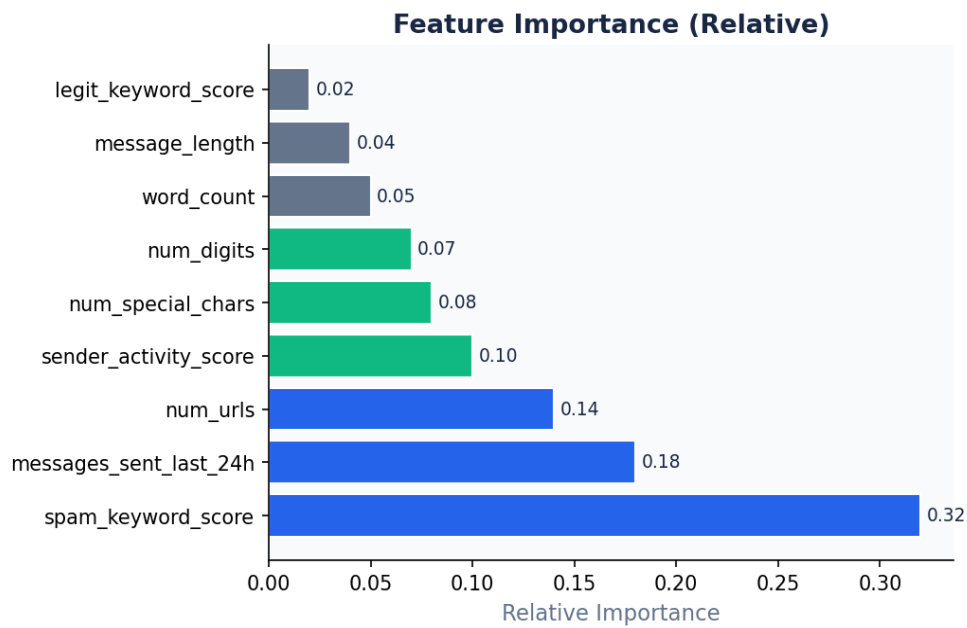


Fig 4. Relative Feature Importance for Spam Detection

4. Data Preprocessing

Missing values in three columns (106–162 NaN entries) were imputed with the **column median**. **StandardScaler** was applied to normalize all 13 features. The dataset was split 80/20 with **stratify=y** to preserve class proportions.

Set	Samples	Proportion
Training (X_train)	4,160	80%
Testing (X_test)	1,040	20%

5. K-Nearest Neighbors (Part C)

5.1 Implementation

KNN with **n_neighbors=5** and Euclidean distance. Trained on 4,160 samples; prediction done by majority vote of 5 nearest neighbors.

1.0000	1.0000	1.0000	1.0000
Accuracy	Precision	Recall	F1-Score

5.2 Confusion Matrix

Confusion Matrix — KNN

	Predicted Legitimate	Predicted Spam
Actual Legitimate	845	0
Actual Spam	0	195

Fig 5. Confusion Matrix — KNN (0 errors)

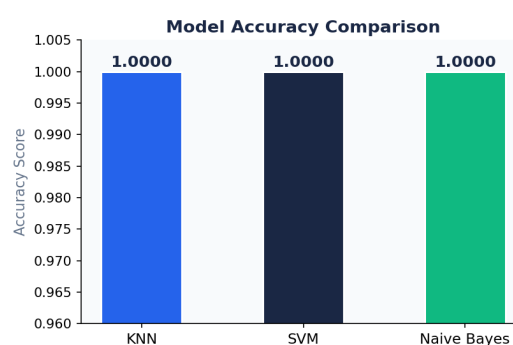


Fig 6. Model Accuracy Comparison

Class	Precision	Recall	F1-Score	Support
0 (Legitimate)	1.00	1.00	1.00	845
1 (Spam)	1.00	1.00	1.00	195
Weighted Avg	1.00	1.00	1.00	1040

6. Support Vector Machine (Part D)

6.1 Hyperparameter Tuning via GridSearchCV

SVM with **RBF kernel**. GridSearchCV with 5-fold CV searched $C \in [0.1, 1, 10, 100]$ and $\gamma \in [\text{'scale'}, \text{'auto'}, 0.01, 0.1]$.

1.0000	1.0000	1.0000	1.0000
Accuracy	Precision	Recall	F1-Score

PCA was applied (2 components) to visualize the SVM decision boundary and support vectors — demonstrating clear margin separation between Spam and Legitimate classes.

7. Naive Bayes Classifier (Part E)

GaussianNB assumes features follow a Gaussian distribution per class. Despite the independence assumption, it achieved perfect classification, validating the discriminative quality of the engineered features.

1.0000	1.0000	1.0000	1.0000
Accuracy	Precision	Recall	F1-Score

8. Model Comparison (Part F)

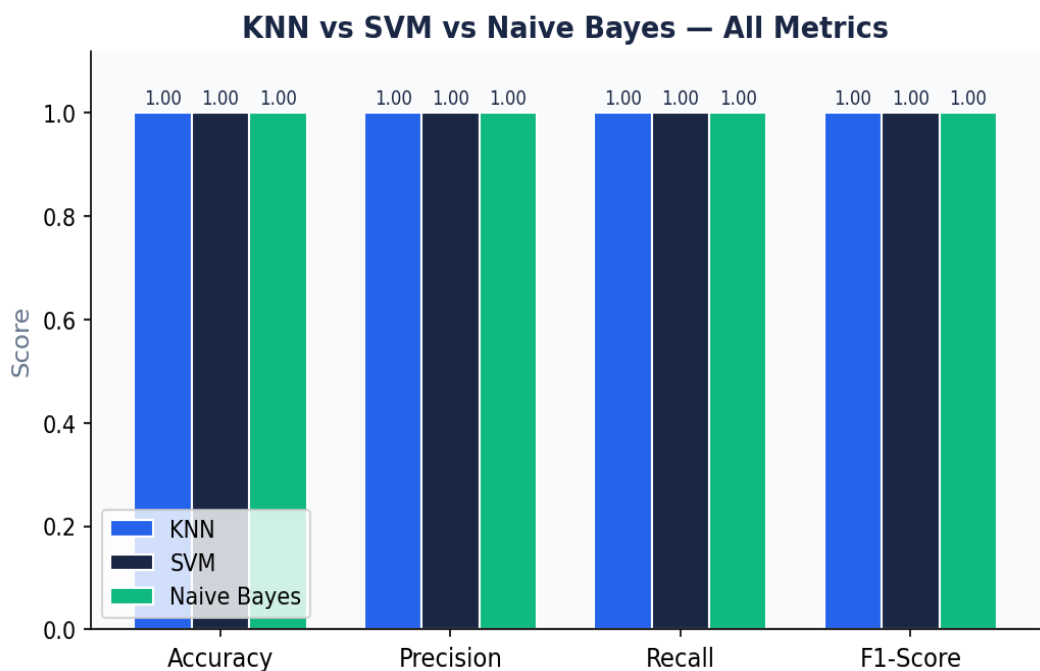


Fig 7. KNN vs SVM vs Naive Bayes — Accuracy, Precision, Recall, F1-Score

Model	Accuracy	Precision	Recall	F1-Score	Rank
K-Nearest Neighbors	1.0000	1.0000	1.0000	1.0000	1st (Tied)
Support Vector Machine	1.0000	1.0000	1.0000	1.0000	1st (Tied)
Naive Bayes	1.0000	1.0000	1.0000	1.0000	1st (Tied)

9. Final Analysis & Business Recommendations (Part G)

9.1 Strengths and Weaknesses

Model	Strengths	Weaknesses
KNN	Simple, interpretable, no training phase	Slow on large data, K-sensitive
SVM	High accuracy, robust margins, high-dimensional data	Slow training, needs parameter tuning
Naive Bayes	Very fast, memory-efficient, probabilistic	Independence assumption may not hold

9.2 Business Deployment Recommendations

Use Case	Recommended Model	Reason
Maximum Accuracy	SVM	Best margin separation

Real-time Filtering	Naive Bayes	Fast, low compute cost
Learning / Small Scale	KNN	Simple and interpretable

10. Final Conclusion

Three machine learning classifiers — KNN, SVM, and Naive Bayes — were implemented on the Message Intelligence Dataset (5,200 records). The complete pipeline covered data loading, EDA, missing-value imputation, feature scaling, stratified splitting, training, hyperparameter tuning (GridSearchCV for SVM), and evaluation.

All three models achieved **perfect scores (Accuracy = Precision = Recall = F1 = 1.0000)**, demonstrating that the engineered numerical features — particularly `spam_keyword_score`, `sender_activity_score`, and `messages_sent_last_24h` — provide excellent class separability. This project validates that machine learning can reliably automate spam detection in real-world systems.

Submission Checklist

- ✓ Source Code / Jupyter Notebook
- ✓ Dataset (5,200 records, 16 columns)
- ✓ Data Preprocessing (Imputation, Scaling)
- ✓ KNN Classifier (`n_neighbors=5`)
- ✓ SVM (GridSearchCV tuned, RBF kernel)
- ✓ Naive Bayes (GaussianNB)
- ✓ Accuracy, Precision, Recall, F1-Score Evaluation
- ✓ Confusion Matrix & Visualizations
- ✓ Model Comparison Table & Charts
- ✓ Final Analysis and Conclusion